FlagPrism: Compiler-Assisted Fine-Grained Profiling Across AI Accelerators (Operational Systems)

Anonymous Authors

Abstract

[TODO: Write the abstract after confirming the study findings, system mechanisms, verified hardware coverage, and operational experience. State measured results only after experiments; the working title is provisional.]

1 Introduction

Unified kernel programming frameworks allow operator developers to reuse program logic across artificial intelligence (AI) accelerators. For example, FlagTree extends the Triton ecosystem to multiple accelerator backends [1]. We consider NVIDIA as the optimization starting point: developers port implementations and configurations tuned there to other accelerators. Obtaining high performance still requires understanding how a target executes that logic: where computation, data movement, and synchronization consume time; whether these activities overlap; and how tiling, layout, and pipeline choices change execution. These profiling questions recur across devices, even when their answers require different execution objects and measurement mechanisms. A useful profiler must connect the resulting observations to the operator implementation being optimized.

Architecture diversity appears at more than one level. Graphics processing units (GPUs) with similar single-instruction, multiple-thread (SIMT) execution models can differ in thread organization, local-memory structure, resource limits, and asynchronous facilities [6, 7]. Accelerators with dedicated computation and data-movement pipelines introduce further differences in how operations are scheduled and synchronized [4]. Such differences affect profiling as well as optimization. A source region can be decomposed, fused, reordered, or pipelined during compilation; timing the issue of an asynchronous operation need not measure its execution or completion. A common interface or output format therefore does not, by itself, determine which execution events should be observed, where instrumentation belongs, or how records should be related back to a program region.

Existing work supplies important foundations. KPerfIR integrates fine-grained profiling into compiler intermediate representation (IR) and analyzes intra-kernel execution and overlap [2]; Neutrino supports programmable instruction-level probes [3]. Proton combines vendor measurement interfaces and instrumentation with multi-level program context [9], while PASTA provides a modular analysis framework for accelerator workloads [5]. These capabilities motivate a concrete question for a broader collection of execution backends: which program mappings, instrumentation semantics, and measurement assumptions can be reused, and which must change when the target architecture changes? Our discussion of Proton and KPerfIR uses the information supplied by their NVIDIA implementations as the reference. The study identifies the additional target-specific information needed to diagnose porting losses.

This paper investigates how fine-grained profiling can diagnose and help recover performance lost after porting. The resulting observations must explain the logical operator region or execution behavior being optimized. We examine three sources of difficulty. First, compilation changes the relationship between logical regions and execution instances. Second, instrumentation must respect the target's execution semantics, including the distinction between submission, execution, and waiting. Third, timestamps, recording mechanisms, and vendor observations must be reconstructed into program-associated results without hiding architecture-specific detail. The study examines targets with similar and different execution models.

We place an empirical study before the system design. Starting from NVIDIA-tuned operators, it examines performance after porting and the portion recoverable through target-specific changes. Two cases connect execution-level causes to fine-grained diagnosis, implementation changes, and uninstrumented performance recovery. They expose information that ordinary timing and source-region observations leave unresolved. The first case keeps a cooperative row-program organization while NVIDIA warps and GCU vector work-items receive different mappings; the second unfolds one

fused tile into Ascend AIC/AIV producer–consumer actors. We do not classify GCU as NVIDIA-like SIMT hardware. Target-local configuration changes recover 1.62× and5.31× performance in the studied cases.

We investigate FlagPrism, a profiler whose system boundary includes the profiling-related compilation mechanisms in FlagTree, together with runtime collection, event association, and analysis. Its intended service is to lower common profiling requests into target-appropriate observations and reconnect those observations to the operator program. It provides profiling services to operator developers and optimization tools. The compiler–profiler boundary and its concrete mechanisms will be defined from the study rather than fixed in advance. **[TODO:** Finalize whether profiling intent requires a new dialect, extensions to existing IR, metadata and passes, or a combination. Distinguish inherited compiler/profiler functionality, implemented extensions, and proposed work.]

The planned contributions are:

- An empirical study of performance loss after porting NVIDIA-tuned operators, with execution-level diagnoses and targeted recovery under cooperative program and producer–consumer organizations (Section 3).
- Compiler–profiler mechanisms for program mapping, target-specific instrumentation, and program-associated reconstruction. **[TODO:** Name and substantiate only the mechanisms actually implemented.]
- A system evaluation and operational account of profiling-guided operator optimization, including measurement fidelity, perturbation, unsupported observations, and unsuccessful diagnoses. **[TODO:** Supply actual case studies and usage records; no deployment or performance claim is established yet.]

2 Background and Profiling Scope

2.1 Operators, Backends, and Execution Models

[TODO: Define the operator programming model, supported input/configuration space, backend compiler versions, and concrete chip models. Separate vendor, architecture generation, execution model, and software stack. Describe NVIDIA, Iluvatar, and Ascend at verified granularity for the study; introduce additional vendors only as needed for system evaluation.]

2.2 From Program Regions to Device Execution

A logical region is a developer-visible part of an operator. An execution instance is the concrete work associated with a particular invocation and configuration. Their correspondence may be one-to-many or many-to-one. **[TODO:** Choose the exact identity and provenance model after inspecting backend IR; define treatment of fusion, duplication, loops, and overlapping regions.]

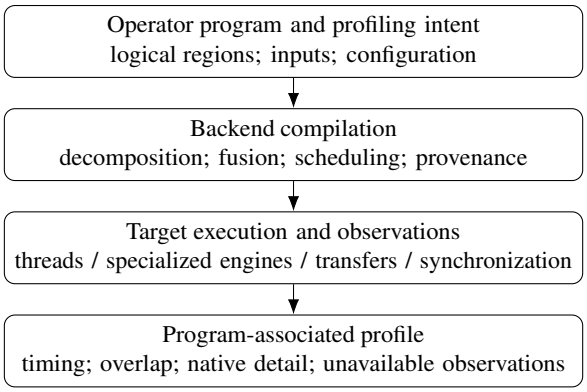


Figure 1: Conceptual profiling scope. The branches illustrate possible mappings, not measured schedules or a claim about a particular chip. The study will replace them with compiler-derived and runtime-validated examples.

2.3 Measurement Semantics

[TODO: Define host submission, device execution, asynchronous completion, waiting, overlap, timestamp domains, and estimated versus measured quantities. State which quantities cannot be observed on each tested environment.]

3 Study: Diagnosing Performance Loss after Porting

Story. NVIDIA-tuned implementation → correct target port with different performance → execution-level cause → backend-specific diagnosis → targeted change and recovery.
Setup. Six operators; three shapes and two frozen NVIDIA configurations per shape; RTX PRO 6000, GCU300, and Ascend 910B4-1. The source implements FP16-input/FP32-accumulation GEMM and fused GEMM+bias+GELU; softmax, layer normalization, reduction, and transpose use FP32. Identical input bytes → CPU-reference correctness checks → saved compiled mappings → native kernel/task intervals. All execution, tuning, and collection run in server Docker containers. Backend compiler/SDK versions differ; clocks and exclusive device use are not fixed. Results characterize these implementations and software stacks.
Optimization starting point. A bounded NVIDIA search covers 1,152 configurations across 20 operator/shape combinations, including the two long-K matrix cases; 1,112 pass and 40 exceed shared-memory limits. Resident CUDA-graph throughput → shortlist three distinct binaries → three native batches of 30 calls per candidate → freeze the native-selected

configuration. The scope includes GEMM warp requests 4/8 and row/transpose requests 1/2/4/8; this is not exhaustive or vendor-best tuning. Independent post-freeze captures supply the plotted performance distributions and case comparisons.

3.1 Performance Differences across an Operator Set

Coverage. Compute, fusion, reductions, and data movement → different shapes and source configurations → Figure 2. Configuration0 is the frozen NVIDIA selection; configuration 1 is the next distinct native-shortlisted configuration. Ports retain the requested configurations and mathematical operations. Each backend contributes 108 captures and 3,240 positive native intervals; all 324 captures pass input/configuration and numerical checks.

Interpreting large gaps. Duration alone does not distinguish hardware capability from an inefficient implementation path. For example, GCU GEMM 512^3 with the transferred 32^3 tile and four work-items takes approximately 94.74 ms; an independent device-stream measurement gives 94.76 ms. The compiled per-item matrix is 16×16, with zero reported 2D instruction share. At the same grid, one work-item produces a 32×32 local matrix, positive 2D share, and 268.53μs. This sanity check identifies a software-path contribution; it is not a chip-peak comparison or a universal tile threshold. A 16×32 control also reports 2D activity, so local shape alone is not a universal threshold. This check qualifies the overview; the two main cases below provide more direct program-level diagnoses and controlled adjustments.

3.2 Case A: Cooperative Reduction Mapping

Scope. One row program and cooperating execution groups are common to both ports; NVIDIA warps and GCU vector work-items have different capacities and instruction organization. We compare this shared program organization, without classifying GCU300 as NVIDIA-like SIMT hardware.

Observation. Softmax 127×769, one row per program. NVIDIA selects four warps: independent native medians are 1.152μs for four and 1.248μs for one. The exact GCU port instead improves from approximately 115.06 to 71.12μs when the corresponding request changes four work-items to one, with grid 127 and the input fixed: 1.62× target-local speedup.

Diagnosis. Timing ranks configurations but does not explain how cooperation is organized on the target. Actual lowering exposes ownership and partial exchange; this suggests testing fewer vector work-items, with the prediction that a coarser per-item reduction can improve the port. Same request → different ownership. Actual NVIDIA indexing stripes columns across warps, assigning 193/192/192/192 useful elements; GCU assigns contiguous slices of 256/256/256/1. GCU four-item lowering uses shared partial exchange and two barriers;

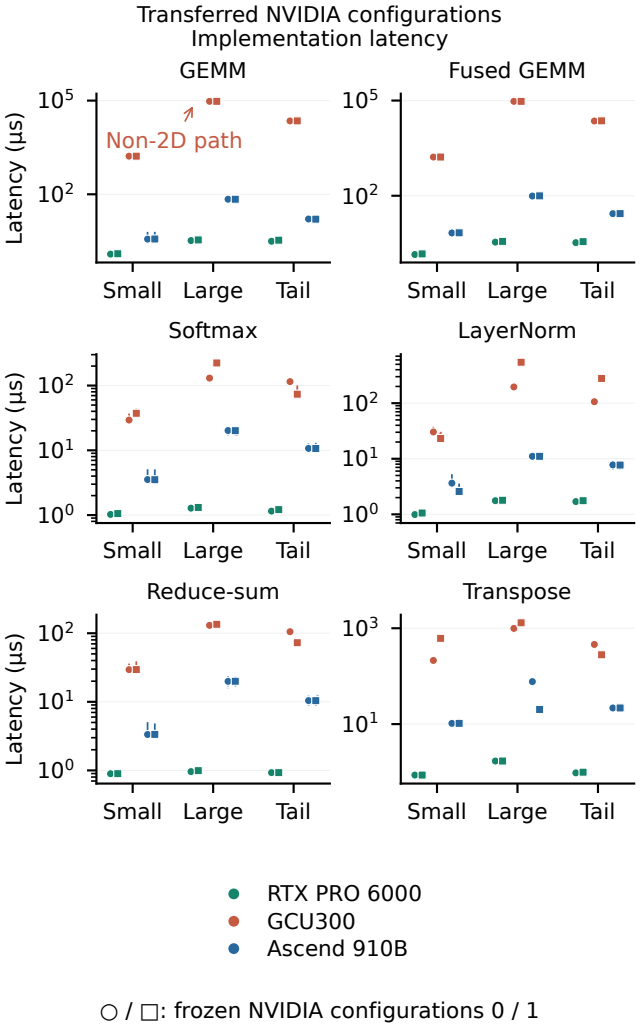


Figure 2: Transferred NVIDIA configurations: native implementation latencies of the ported implementations. Each shape/configuration has three batches of 30 prewarmed calls per backend. Small dots: calls; circle/square: configuration 0/1 medians; vertical spans: descriptive min–max, not confidence intervals. Matrix shapes are 128^3, 512^3, $257 \times 193 \times 320$; row shapes 32×256, 256×1024, 127×769; transpose shapes 256^2, 1024^2, 513×257. Configurations are selected on NVIDIA and requested unchanged on targets. Large GCU matrix values reflect the implementation path discussed in the text; this is not a hardware-peak ranking.

one-item lowering owns the full row and removes the exchange. Original-binary replay counters show the SDK 1D instruction-cycle share rising from 0.841% to 2.991%. The share uses SDK total cycles as its denominator; it is a companion execution characteristic, not GPU occupancy or an isolated explanation of the speedup. Static element counts do not measure utilization or prove padding as the sole cause.

Controlled adjustment. Hold source math, input, row group-

ing, and grid fixed → change only the worker request → verify the changed mapping and three independent 30-call native batches. A two-item control keeps both barriers but reaches approximately 73.65μs; barrier presence alone therefore does not explain the four-item slowdown. Ownership, local/shared resources, and worker-dependent synchronization costs change together. The adjustment is validated; their individual cost contributions remain unseparated.

3.3 Case B: Producer–Consumer Granularity

Observation. Fused GEMM $512 \times 512 \times 4096$, a longer-$K$ shape outside the overview, exposes producer–consumer behavior. The frozen NVIDIA implementation uses 32^3 tiles, four warps, and stage request 4; independent native latency is 17.728μs. Its Ascend port takes 310.27μs. Each logical tile becomes one AIC producer and two AIV consumers, connected by a global-memory workspace and readiness event 0. The stage request enables the backend's multibuffering; it does not mean four physical buffers.

Diagnosis. Long output-stage time → computation or readiness? In SDK program 0 replays, AIV0 wait0 activity is 46.72μs while vector activity is 0.293μs. Increasing only B_K from 32 to 256 leaves the saved AIV LLVM byte-identical, but reduces producer Cube commands 128→16, MTE2 activity 47.52→23.81μs, and AIV0 wait0 46.72→26.40μs; vector activity stays 0.293μs. These activities may overlap and are not added to reconstruct wall time. Sparse scalar-clock records expose iteration IDs, issue gates, and L1 buffers. Joint source/native captures uniquely associate all three program 0 actors with one invocation and empirically validate a shared reported SYS epoch. Figure 4(b) aligns producer control and consumer wait brackets from each actual call; bars measure source control, not engine occupancy. Program 0 is one logical tile with three actors; these local observations diagnose a dependency, while whole-launch time covers all programs. They neither represent every instance nor reconstruct a launch critical path.

Staged recovery. Producer granularity first: $32 \times 32 \times 32 \rightarrow 32 \times 32 \times 256$ → 310.27→164.12μs, with identical output tiling and logical input payload. The compiled instance and source-loading counts suggest output tiling next: $64 \times 64 \times 256$ →58.45μs. Logical programs 256→64 and source-requested A/B payload 128→64 MiB; physical DRAM traffic is not inferred from this count. The larger-tile case does not materially lower program 0 wait further: the second gain comes with reduced instance cardinality and source-requested loading; their separate causal contributions are not identified. Held-out randomized native batches and CPU checks confirm the two-step 5.31× recovery. Additional B_K and unroll explorations, including non-improvements, remain in the artifact with their distinct configurations; they are not all controls of the final frozen implementation.

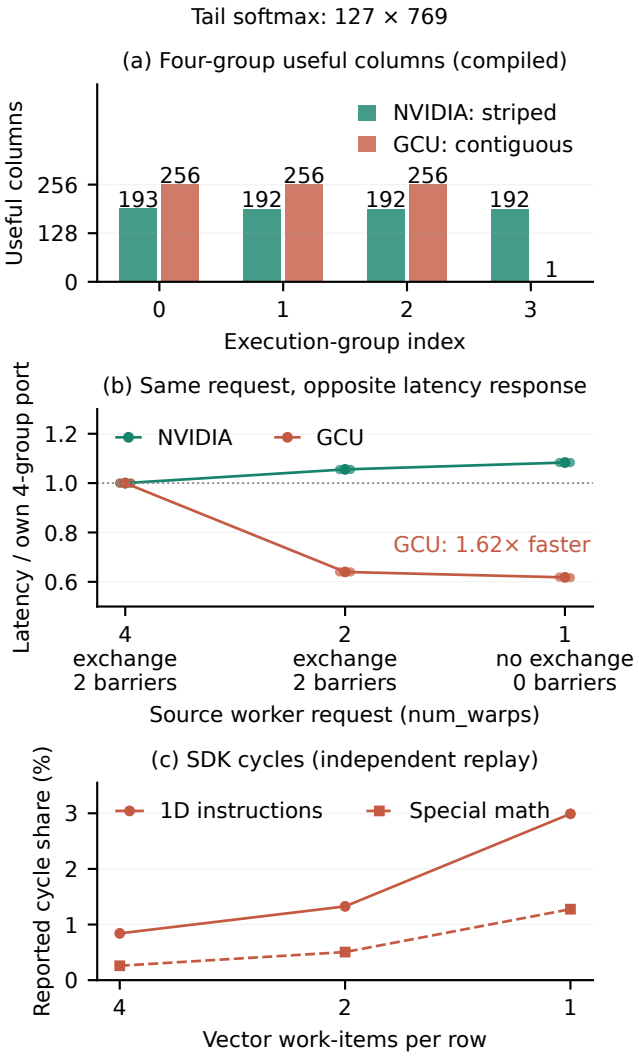


Figure 3: Tail softmax. (a) Compiler-verified useful-column assignment to 32-lane NVIDIA warps versus GCU vector work-items; capacities differ. (b) Uninstrumented latency normalized to each backend's four-group version; dots denote three batch medians, 30 calls per batch. (c) SDK-defined per-SIP cycle shares from original-binary TCU replays, 21 calls including six initial/warmup calls, 24 SIP instances per record. Replay counters are separate captures and are not confidence intervals or exclusive wall-time fractions. The two-item control shows that barrier presence alone does not explain the slowdown; synchronization cost itself is not isolated.

3.4 Information Needed to Diagnose the Loss

Case A. Total latency shows a target-side gap; source-level grouping alone does not reveal striped versus contiguous ownership, shared exchange, or vector resource activity. Compiled mappings → fewer-item hypothesis → controlled worker adjustment; per-SIP counters characterize the changed path.

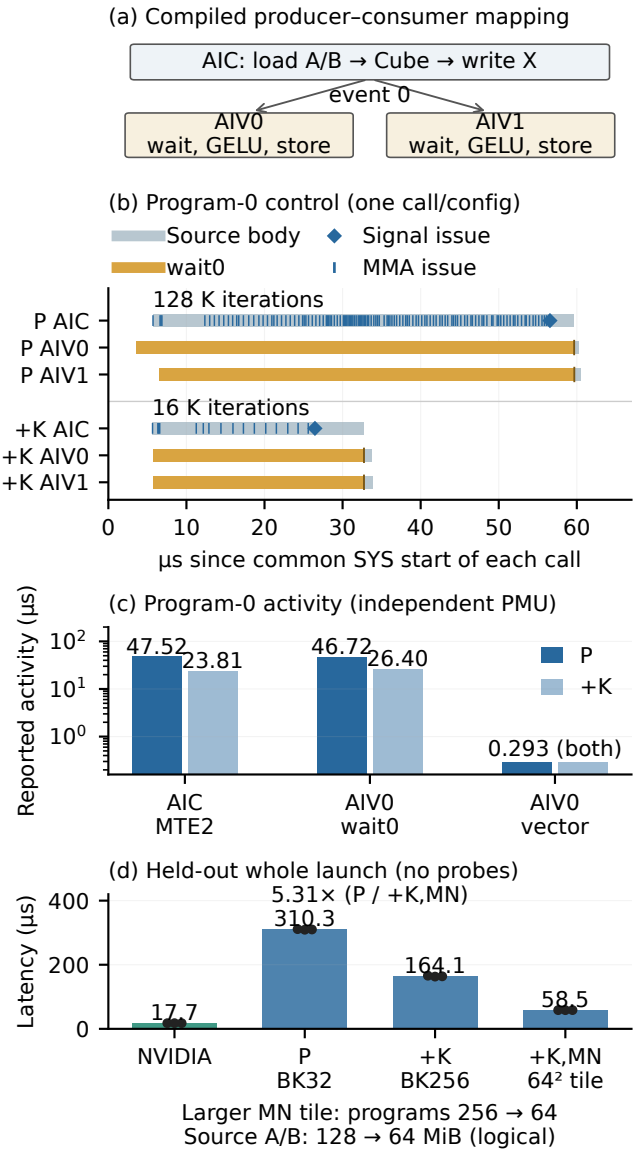


Figure 4: Fused-case diagnosis and recovery. P: NVIDIA configuration port; +K: $B_K = 256$; +K,MN: also $B_M/B_N = 64$. (a) Compiled dependencies. (b) One actual joint capture per configuration, program 0; warm-up 0 SDK replay. Uniquely matched native windows validate the shared reported SYS epoch. Bars: source body (gray), scalar wait0 brackets (gold); ticks: MMA-helper issue; diamond: signal issue. These are control positions, not hardware busy spans. (c) Independent PMU replays: activity totals may overlap and have different cache/invocation conditions from (b). (d) Held-out native latency, 90 calls/configuration; dots: three batch medians. Sparse prewarmed whole-task perturbation ratios 1.0045/0.9975/1.0071 apply to separate captures, not zero-overhead guarantees for (b).

Case B. A long output region does not identify the activity responsible. Actor identity + readiness event + producer command granularity → distinguish waiting from vector execution → improve the producer before changing the consumer.

Profiling gap. Proton and KPerfIR supply NVIDIA profiling information, including fine-grained regions [2, 9]. That information does not directly specify GCU vector ownership or Ascend AIC/AIV readiness dependencies. Explaining these cases requires backend-specific execution information associated with the source operator, beyond ordinary kernel or region durations. Timing-only search can still find faster configurations; the demonstrated gap is diagnostic explanation, not an impossibility of optimization without us. Our generic probes are not executions of those tools.

Transition to the design. Source operator/region → actual execution instances and native activities → optimization feedback. This motivates profiling services in FlagPrism with compilation support from FlagTree: preserve backend detail and its program association. The study validates research observations and configuration changes; a general profiling IR or production mapping mechanism must be substantiated separately in the design and evaluation.

4 Compiler-Assisted Profiling Design

[TODO: Tie each mechanism to the information needed for the diagnoses and adjustments in Sections 3.2–3.4. Use recurring mapping work to motivate a common abstraction only when the cases support it. Freeze the system boundary before claiming design novelty.**]**

4.1 Profiling Intent and Backend Contracts

[TODO: Define requests, observations, supported capabilities, and the native extension mechanism. Decide between a new profiling IR and extensions to existing IR/metadata. Specify semantics rather than only an API.**]**

4.2 Program Mapping Through Compilation

[TODO: Explain logical-region identity, transformation provenance, one-to-many and many-to-one relationships, and attribution after fusion and scheduling.**]**

4.3 Execution-Aware Instrumentation

[TODO: Describe instrumentation placement and lowering for each execution model. Distinguish issue, completion, and waits; state unsupported quantities. Explain how instrumentation interacts with optimization and synchronization.**]**

4.4 Collection and Reconstruction

[TODO: Define timestamp calibration, buffering, event association, lost-record handling, concurrency, and the separation of measurements from static or derived relationships. Preserve backend-specific observations.**]**

5 Implementation

[TODO: Record pinned FlagTree/FlagPrism commits and backend toolchains. Document compiler passes, instrumentation runtime, vendor collectors/importers, and profile outputs. Quantify shared versus backend-specific implementation without treating line count as proof of abstraction quality.**]**

5.1 Backend Integration and Capability Status

[TODO: Use a per-chip, per-SDK acceptance matrix. Distinguish an API skeleton, a real capture, successful program association, and fidelity validation. A readable report or passing smoke test does not establish full support.**]**

5.2 Limitations and Failure Handling

[TODO: Specify unavailable clocks/counters, permissions, replay constraints, unsupported IR transformations, ambiguous matches, and explicit degradation.**]**

6 System Evaluation

The study in Section 3 characterizes the problem. This section will evaluate the proposed mechanisms on separate or held-out tasks and additional backends beyond the finalized Study devices, initially NVIDIA, Enflame, and Ascend. **[TODO:** Freeze study/evaluation separation and avoid selecting evaluation cases only after seeing favorable study results.**]**

6.1 Measurement and Attribution Fidelity

[TODO: Validate event coverage, region association, timestamp accuracy, and reconstruction against independent reference observations and controlled microbenchmarks. State when a reference is not independent.**]**

6.2 Overhead and Execution Perturbation

[TODO: Report per-mode and per-device latency, memory/register use, trace volume, processing cost, and changes to compiled schedules. Check bottleneck/ranking changes caused by instrumentation.**]**

6.3 Profiling-Guided Operator Optimization

[TODO: For each case record the initial hypothesis, additional observation, intervention, code change, correctness check, and profiler-disabled performance. Compare against native tools and reasonable target-local implementations.**]**

6.4 Mechanism Ablations and Backend Extension

[TODO: Remove transformation provenance, execution-aware placement, or native observation detail separately. Show what information or correctness is lost. Measure integration changes for a backend added after common analyses are frozen.**]**

6.5 Additional Backends

[TODO: Validate actual supported modes on Moore Threads, Iluvatar, MetaX, and other available devices. Report per-backend fidelity, overhead, native detail, and limitations rather than extending the three-backend Study.**]**

7 Operational Experience and Lessons

[TODO: Provide actual adoption context, duration, operators, developer workflows, profiling sessions, resolved/unresolved issues, maintenance burden, and examples of failed hypotheses. The project has not yet supplied bulk usage evidence.**]**

7.1 Lessons Across Similar and Different Execution Models

[TODO: Derive lessons from cases rather than restating architecture descriptions. Discuss which mechanisms were reusable and where the abstraction broke.**]**

7.2 Scope and Threats to Validity

[TODO: Discuss device coverage, compiler/SDK confounding, finite search spaces, measurement perturbation, missing references, and application generalization.**]**

8 Related Work

Compiler and instruction-level profiling. KPerfIR [2], Neutrino [3], and Proton [9] provide compiler, probe, and multi-level profiling foundations. **[TODO:** Compare concrete supported mechanisms and versions; do not attribute common-counter-only limitations to these systems.**]**

[TODO: Draw the final compiler–runtime–analysis architecture, marking inherited components and the profiling-specific extensions introduced by this work.]

Figure 5: System architecture placeholder; mechanisms and boundaries remain to be determined.

Context and accelerator analysis. DeepContext [8] connects program contexts across the deep learning stack, and PASTA [5] supplies modular accelerator analysis. [TODO: Complete comparisons with HPCToolkit, XProf, RL-Scope, Hotline, Snoopie, DrGPUM, and TenProf using the existing-works survey and primary papers. Separate broader workload diagnosis from our operator profiling scope.]

9 Conclusion

[TODO: Summarize established study findings, implemented mechanisms, measured benefits, and limitations after the empirical work is complete.]

References

[1] FlagOS. FlagTree: A unified compiler for multiple ai chip backends. Project repository. Accessed October 12, 2026. https://github.com/flagos-ai/FlagTree.

[2] Yue Guan, Yuanwei Fang, Keren Zhou, Corbin Robeck, Manman Ren, Zhongkai Yu, Yufei Ding, and Adnan Aziz. KPerfIR: Towards a open and compiler-centric ecosystem for GPU kernel performance tooling on modern AI workloads. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 205–220. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/guan.

[3] Songlin Huang and Chenshu Wu. Neutrino: Fine-grained GPU kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 331–355. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/huang-songlin.

[4] Huawei. AI Core architecture. Ascend CANN documentation. Accessed October 12, 2026. https://www.hiascend.com/doc_center/source/en/canncommercial/800/opdevg/tbeaicpudevg/atlasopdev_10_0008.html.

[5] Mao Lin, Hyeran Jeon, and Keren Zhou. PASTA: A modular program analysis tool framework for accelerators. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/28/PASTA-A-Modular-Program-Analysis-Tool-Framework-for-Accelerators.

[6] Moore Threads. MUSA hardware architecture. MUSA SDK 4.3.x programming guide. Historical version; architecture statements apply to its documented devices. Accessed October 12, 2026. https://docs.mthreads.com/musa-sdk/version-4.3.x/programming_guide/Chapter02/.

[7] NVIDIA. Hopper tuning guide. CUDA documentation. Accessed October 12, 2026. https://docs.nvidia.com/cuda/archive/13.2.0/hopper-tuning-guide/index.html.

[8] Qidong Zhao, Hao Wu, Yueming Hao, Zilingfeng Ye, Jiajia Li, Xu Liu, and Keren Zhou. Deepcontext: A context-aware, cross-platform, and cross-framework tool for performance profiling and analysis of deep learning workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 48–63, 2025. https://arxiv.org/abs/2411.02797.

[9] Keren Zhou, Tianle Zhong, Hao Wu, Jihyeong Lee, Yue Guan, Yufei Ding, Corbin Robeck, Yuanwei Fang, Jeff Niu, and Philippe Tillet. Proton: Towards multi-level, adaptive profiling for triton. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/23/Proton-Towards-Multi-level-Adaptive-Profiling-for-Triton.